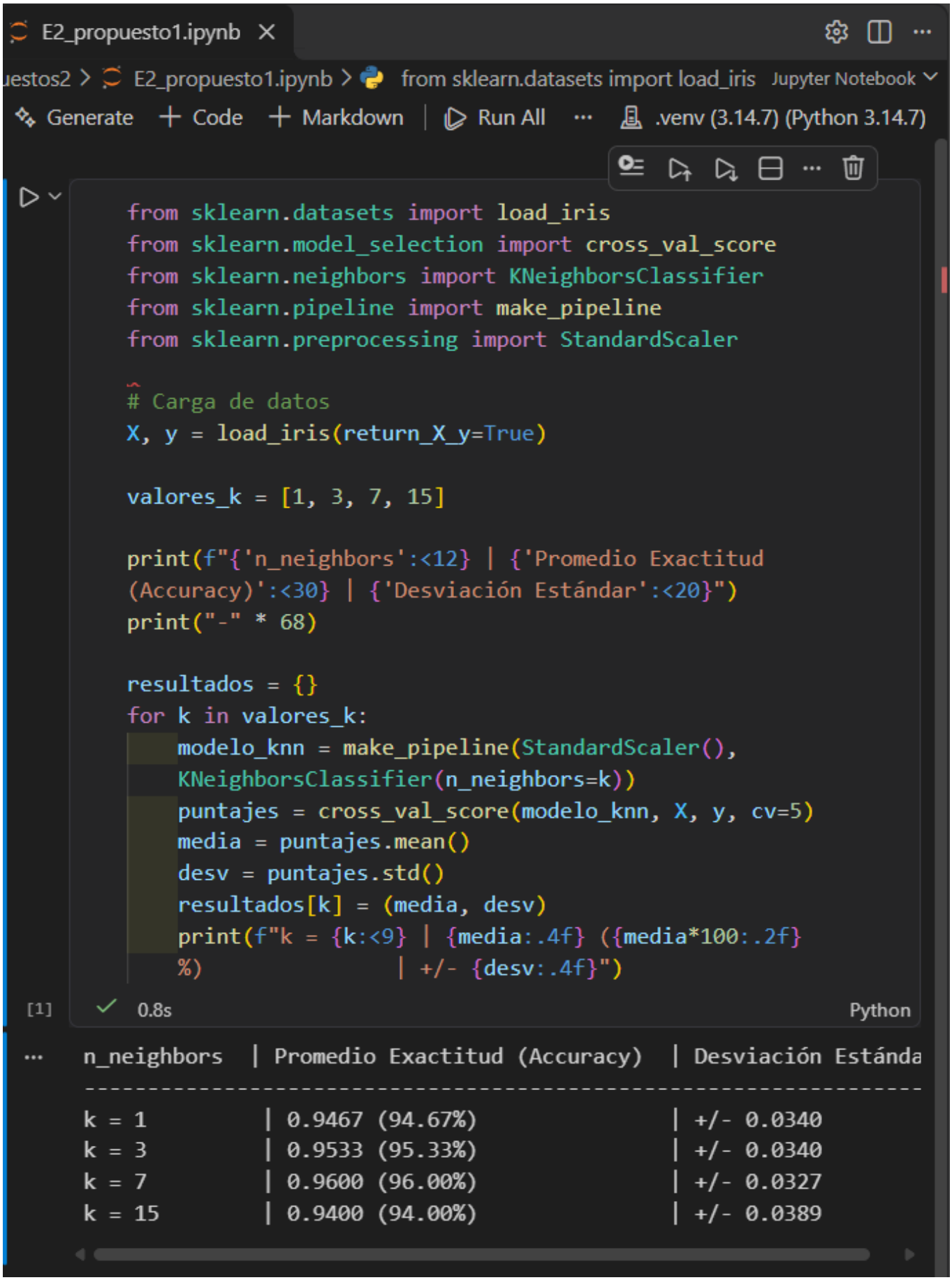
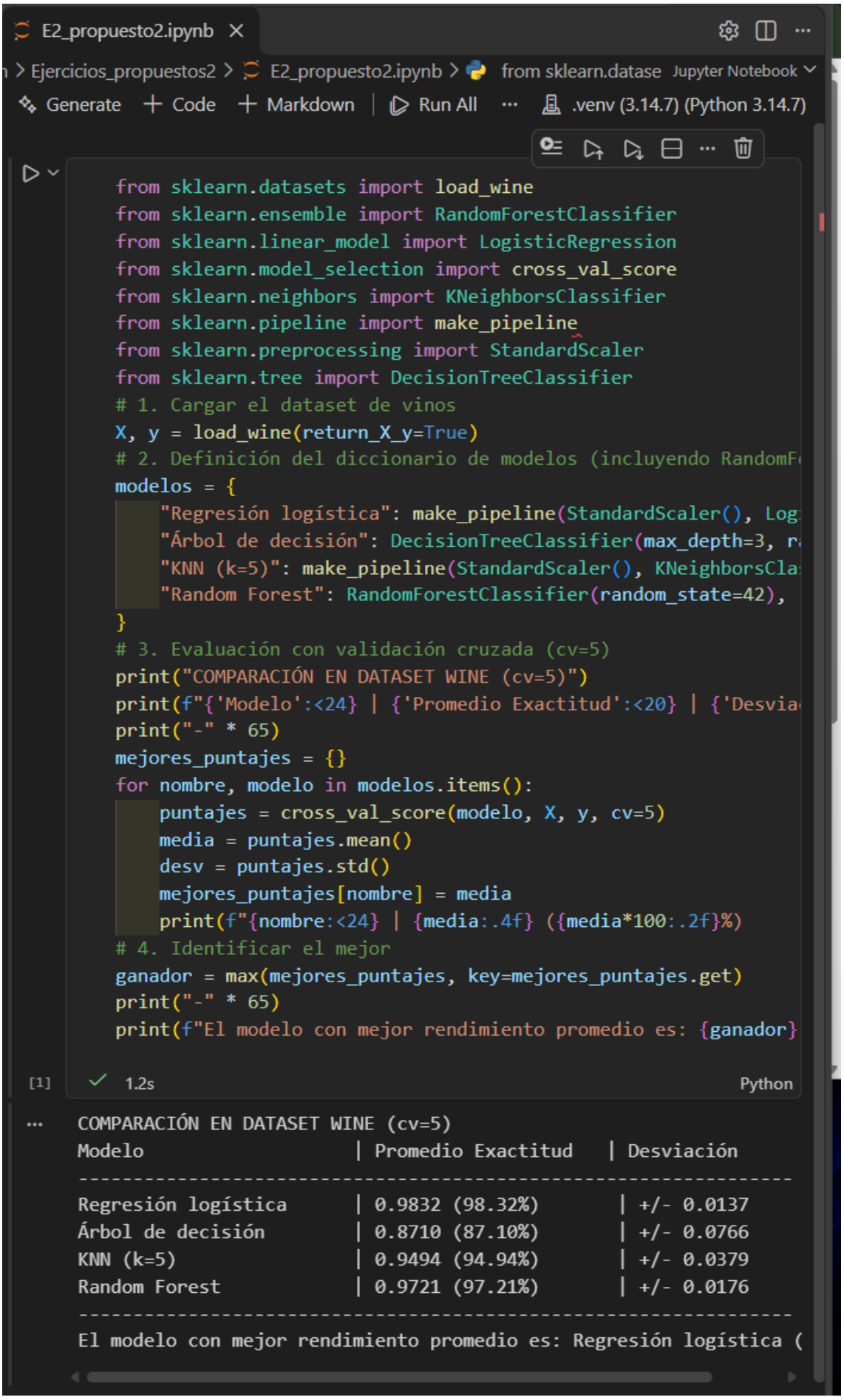
CLASE 2: TAREA 2

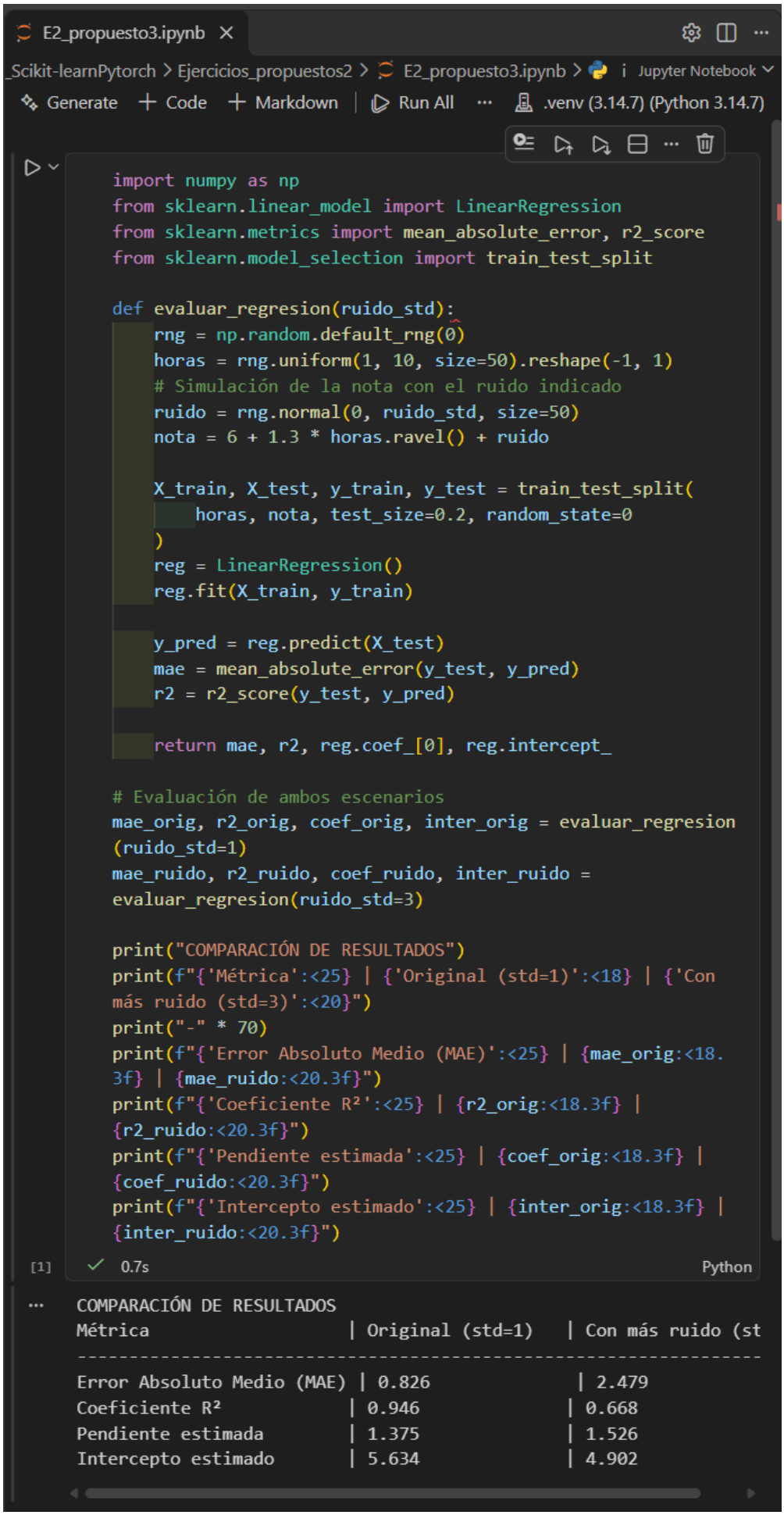
1. Scikit-learn: en el Paso 11 prueba con n_neighbors igual a 1, 3, 7 y 15. Anota la exactitud de cada uno en una tabla y explica cuál elegirías.



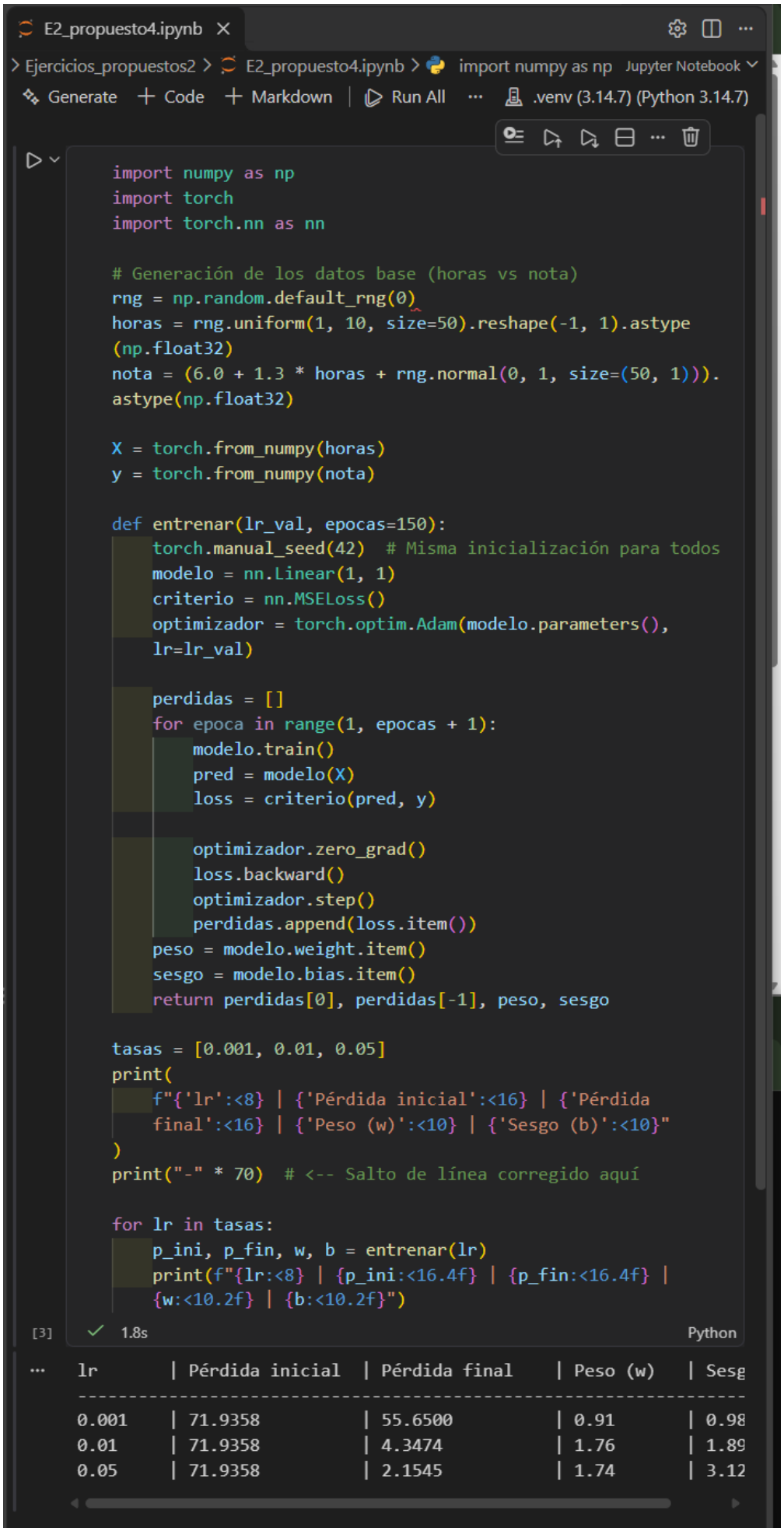
2. Scikit-learn: carga el dataset load_wine() y repite el Paso 12 agregando el modelo RandomForestClassifier(random_state=42). ¿Cuál obtiene el mejor promedio?



3. Scikit-learn: modifica el Paso 13 para que los datos tengan más ruido (rng.normal(0, 3, ...)). ¿Cómo cambian el MAE y el R^2?



4. PyTorch: en el Paso 15 cambia lr a 0.001 y a 0.05. Describe qué pasa con la pérdida y con los valores aprendidos.



5. PyTorch: en el Paso 16 agrega una segunda capa oculta de 8 neuronas con ReLU. Compara la exactitud con la red original.

E2_propuesto5.ipynb

IA_Guia02_Scikit-learnPytorch > Ejercicios_propuestos2 > E2_propuesto5.ipynb > import torch

Jupyter Notebook

Generate + Code + Markdown | Run All Restart Clear All Outputs ...

.venv (3.14.7) (Python 3.14.7)

```python
import torch
import torch.nn as nn
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
# Fijar semilla general
torch.manual_seed(42)
# 1. Preparar datos
X, y = load_iris(return_X_y=True)

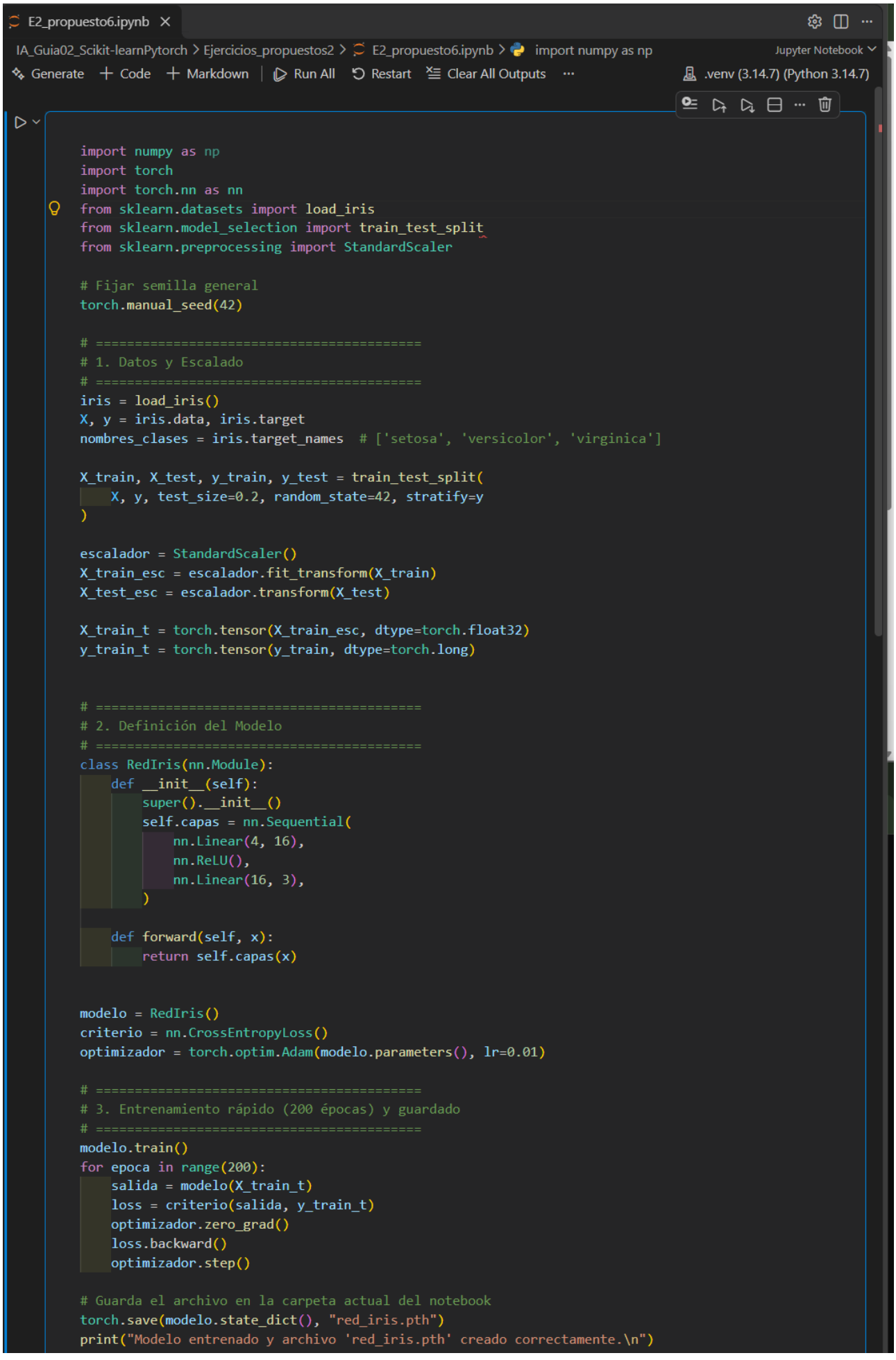
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
esc = StandardScaler()
X_train = esc.fit_transform(X_train)
X_test = esc.transform(X_test)
X_train = torch.tensor(X_train, dtype=torch.float32)
X_test = torch.tensor(X_test, dtype=torch.float32)
y_train = torch.tensor(y_train, dtype=torch.long)
y_test = torch.tensor(y_test, dtype=torch.long)
# 2. Definición de ambas redes
# Red original: 1 capa oculta (16 neuronas)
class RedOriginal(nn.Module):
    def __init__(self):
        super().__init__()
        self.capas = nn.Sequential(
            nn.Linear(4, 16),
            nn.ReLU(),
            nn.Linear(16, 3),
        )
    def forward(self, x):
        return self.capas(x)
# Red modificada: 2 capas ocultas (16 y 8 neuronas)
class RedModificada(nn.Module):
    def __init__(self):
        super().__init__()
        self.capas = nn.Sequential(
            nn.Linear(4, 16),
            nn.ReLU(),
            nn.Linear(16, 8),  # Segunda capa oculta añadida
            nn.ReLU(),         # Activación ReLU añadida
            nn.Linear(8, 3),   # Capa de salida conectada a las 8 neuronas
        )
    def forward(self, x):
        return self.capas(x)
# 3. Función de entrenamiento y evaluación
def entrenar_y_evaluar(modelo, nombre):
    torch.manual_seed(42)
    criterio = nn.CrossEntropyLoss()
    optimizador = torch.optim.Adam(modelo.parameters(), lr=0.01)
    for epoca in range(1, 201):
        modelo.train()
        salida = modelo(X_train)
        perdida = criterio(salida, y_train)
        optimizador.zero_grad()
        perdida.backward()
        optimizador.step()
    modelo.eval()
    with torch.no_grad():
        preds = modelo(X_test).argmax(dim=1)
        acc = (preds == y_test).float().mean().item()
    return perdida.item(), acc
```

```python
# 4. Comparación
loss_orig, acc_orig = entrenar_y_evaluar(RedOriginal(), "Original")
loss_mod, acc_mod = entrenar_y_evaluar(RedModificada(), "Modificada (2 capas)")
print("=== COMPARACIÓN DE ARQUITECTURAS ===")
print(f"{'Modelo':<30} | {'Pérdida final (Loss)':<22} | {'Exactitud (Accuracy)':<20}")
print("-" * 78)
print(f"{'Original (4 -> 16 -> 3)':<30} | {loss_orig:<22.4f} | {acc_orig:<20.3f} ({acc_orig*100:.1f}%)")
print(f"{'Modificada (4 -> 16 -> 8 -> 3)':<30} | {loss_mod:<22.4f} | {acc_mod:<20.3f} ({acc_mod*100:.1f}%)")
```

[2] ✓ 0.2s Python

```
=== COMPARACIÓN DE ARQUITECTURAS ===
Modelo                         | Pérdida final (Loss)   | Exactitud (Accuracy)
------------------------------------------------------------------------------
Original (4 -> 16 -> 3)        | 0.0432                 | 1.000                (100.0%)
Modificada (4 -> 16 -> 8 -> 3) | 0.0292                 | 0.967                (96.7%)
```

6. Reto integrador: carga los pesos guardados en red_iris.pth en un nuevo archivo (modelo.load_state_dict(torch.load("red_iris.pth"))) y predice la especie de 3 flores inventadas por ti.



```python
# ==========================================
# 4. Carga de los pesos (cumpliendo el reto)
# ==========================================
modelo_eval = RedIris()
modelo_eval.load_state_dict(torch.load("red_iris.pth", weights_only=True))
modelo_eval.eval()

# ==========================================
# 5. Definir 3 flores inventadas y predecir
# ==========================================
flores_inventadas = [
    [5.0, 3.4, 1.5, 0.2],  # Medidas de Setosa
    [6.0, 2.8, 4.3, 1.3],  # Medidas de Versicolor
    [7.5, 3.2, 6.5, 2.2],  # Medidas de Virginica
]

flores_esc = escalador.transform(flores_inventadas)
flores_tensor = torch.tensor(flores_esc, dtype=torch.float32)

with torch.no_grad():
    salidas = modelo_eval(flores_tensor)
    probabilidades = torch.softmax(salidas, dim=1)
    predicciones = salidas.argmax(dim=1)

# ==========================================
# 6. Salida de resultados
# ==========================================
print(" PREDICCIONES DEL MODELO CARGADO ")
for i, (flor, pred_idx, probs) in enumerate(
    zip(flores_inventadas, predicciones, probabilidades), start=1
):
    clase = nombres_clases[pred_idx.item()]
    confianza = probs[pred_idx].item() * 100
    print(f"Flor {i}: {flor}")
    print(f"  -> Especie: {clase.upper()} (Clase {pred_idx.item()})")
    print(f"  -> Certeza: {confianza:.2f}%\n")
```

[1] ✓ 2.5s Python

```
Modelo entrenado y archivo 'red_iris.pth' creado correctamente.

 PREDICCIONES DEL MODELO CARGADO
Flor 1: [5.0, 3.4, 1.5, 0.2]
  -> Especie: SETOSA (Clase 0)
  -> Certeza: 99.98%

Flor 2: [6.0, 2.8, 4.3, 1.3]
  -> Especie: VERSICOLOR (Clase 1)
  -> Certeza: 99.90%

Flor 3: [7.5, 3.2, 6.5, 2.2]
  -> Especie: VIRGINICA (Clase 2)
  -> Certeza: 100.00%
```